CyberShield AI: A Web-Based Real-Time Cybersecurity Monitoring Dashboard with AI-Assisted Threat Analysis

Sreedevi R P, Vishnu V Nair, Abhi Raj
Varsha C Mohan, Geetha S, Suvarna Dev
Department of Computer Engineering
College of Engineering Chengannur
APJ Abdul Kalam Technological University, Kerala, India

Abstract*— Security operations centers (SOCs) and small IT teams face a widening gap between the accuracy of individual machine-learning threat detectors reported in research and the practical tools available to translate a detection into a timely, understandable, and actionable response. Existing literature on phishing and malicious-URL detection, malware analysis, and human-centered security largely treats these problems in isolation: high offline accuracy is reported, but real-time alerting, explainability, and integration with a usable interface are rarely addressed together. This paper presents CyberShield AI, a web-based cybersecurity monitoring dashboard that aggregates security telemetry, provides on-demand phishing-URL and malware scanning, and uses the Google Gemini API to generate expert, human-readable explanations for detected threats. The system is built around a React/Vite dashboard, a Node.js/Express REST API gateway, and a set of independent backend services, Auth & RBAC, AI Analysis, Threat Intelligence, and Alerts & Webhooks, that call local ML models, the Gemini API, and VirusTotal, with results persisted to Firestore and an audit log store. This paper describes the system's architecture and operational workflow, and positions the design relative to gaps identified in recent phishing detection, malware detection, and human-centered cybersecurity research.

***Keywords*—** *Cybersecurity Dashboard, Phishing Detection, Malware Scanning, Threat Intelligence, Large Language Models, Real-Time Alerting, Security Operations.*

I. INTRODUCTION

Phishing remains the dominant initial-access vector for cyber-attacks, malware continues to evolve through obfuscation and fileless techniques, and organizations of every size must now monitor a growing volume of network and threat-intelligence data. Machine learning and deep learning have substantially improved the accuracy of phishing-URL classification and malware detection, and large language models (LLMs) have recently been applied to generate human-readable explanations alongside raw classification scores. However, individual research systems are typically evaluated offline, on a single task, and in isolation from the alerting and visualization layer that determines whether a security team can act on a detection quickly.

CyberShield AI is designed to close this gap for individual analysts and small teams by combining threat detection, AI-assisted explanation, and real-time visualization within a single dashboard. Rather than proposing a new detection algorithm, the system integrates local ML-based scanning for phishing URLs and malware with Gemini-generated explanations, live dashboard metrics, threat-intelligence lookups, and configurable alerting, so that a flagged threat is immediately visible, explained in plain language, and, where configured, pushed to a team's existing communication channels. This paper describes the system's architecture and its operational workflow, and situates these design choices relative to gaps identified in recent phishing-detection, malware-detection, and human-centered cybersecurity research.

II. RELATED WORK

Recent research on machine-learning and LLM-based phishing and URL detection reports high in-distribution accuracy: gradient-boosted models reach over 90% accuracy with efficient training [1], LLM-based email analysis with generated rationale reaches 99.70% accuracy [2], crowdsourced feedback loops push accuracy to 99.8% [3], and early detection on newly registered and parked domains achieves 96–97% recall and accuracy for a financial institution [4]. In malware detection, multi-channel image-based fusion of static and dynamic features [5] and hybrid CNN-RNN architectures combining static and dynamic analysis [6] outperform single-domain baselines, hybrid static/dynamic/memory forensic analysis improves artifact recovery for fileless malware [7], and a review of 109 publications confirms accuracy in the 93–98.7% range across deep-learning malware detectors while noting adversarial robustness as an open problem [8]. On the human and organizational side, studies show that detection competence, not risk awareness alone, predicts protective behavior [9], that software development organizations face structurally similar categories of cyber risk [10], that security investment can be justified with FAIR-based economic modeling [11], and that feedback-driven training improves threat comprehension [12].

Across this literature, detection accuracy is consistently high, but the reviewed systems are evaluated as standalone classifiers or literature syntheses rather than as components of a real-time, explainable monitoring interface. CyberShield AI is motivated directly by this gap: it does not

attempt to outperform the specialized classifiers described above, but instead integrates local ML-based scanning with Gemini-generated explanation, live dashboard visualization, and webhook-based alerting so that detection results are delivered in a form a human analyst can immediately understand and act on. Table 1 summarizes how each recurring gap identified in the reviewed literature maps onto a specific design decision in the proposed architecture.

III. PROPOSED SYSTEM ARCHITECTURE

CyberShield AI follows a layered client-server architecture, illustrated in Fig. 1.

A. Actors and Presentation Layer

Two actor roles interact with the system: the *User (Analyst)*, who submits URLs and files for scanning and reviews dashboard metrics, and the *SOC Admin (Security team)*, who additionally manages alert thresholds and webhook configuration. Both access the system through a single Web Dashboard built with React and Vite, which provides phishing and malware scanning, network monitoring, threat-intelligence views, alert management, and settings in one responsive interface.

B. API Gateway and Core Services

All dashboard requests pass through a REST API Gateway implemented in Node.js with Express, which routes requests to four core services: an Auth & RBAC Service that authenticates users and enforces role-based access control; an AI Analysis Engine that coordinates phishing and malware inference; a Threat Intel Service that performs intelligence lookups and is invoked directly by the AI Analysis Engine; and an Alerts & Webhook Service that manages severity-based alert dispatch. Structuring these as separate services behind a single gateway keeps the authentication, analysis, intelligence, and notification concerns independently extensible.

C. Model and External Intelligence Layer

The AI Analysis Engine calls two downstream resources: local ML models (a PhishGuard-style classifier and a random-forest model) for fast inference, and the Google Gemini API, which supplies the explainable-AI (XAI) rationale behind each verdict, directly addressing the interpretability gap highlighted by LLM-based detection research [2]. The Threat Intel Service performs lookups against VirusTotal, aggregating results from more than seventy antivirus engines for file-hash and indicator verification.

D. Persistence and Delivery Layer

Session and user data are persisted in Firestore with an in-memory cache for low-latency reads. Every analysis and lookup is recorded in a dedicated audit log store to preserve an investigable trail of system activity. Finally, the Alerts & Webhook Service dispatches severity-categorized notifications to Discord and Slack, closing the loop between detection and the channels a security team already monitors.

IV. SYSTEM WORKFLOW

CyberShield AI operates through four coordinated stages. First, in request handling, a User or SOC Admin interacts with the Web Dashboard, which forwards every scan, lookup, or configuration change to the REST API Gateway as an authenticated API call validated by the Auth & RBAC Service. Second, in AI-assisted analysis, the AI Analysis Engine runs local ML inference (PhishGuard and random-forest models) for a fast verdict and simultaneously queries the Gemini API for an explainable-AI rationale, while the Threat Intel Service performs VirusTotal lookups across more than seventy antivirus engines. Third, in threat management, the Alerts & Webhook Service categorizes detections by severity and dispatches notifications to Discord or Slack so analysts can triage in order of urgency. Fourth, in persistence and audit, session data is cached through Firestore and every analysis is written to the audit log store, preserving a reviewable record of system activity alongside the live dashboard view.

A. Illustrative Scenario: Phishing URL Submission

To make the workflow concrete, consider an analyst who receives a suspicious link and submits it through the dashboard's Phishing Scanner. (1) The Web Dashboard sends the URL to the REST API Gateway as an authenticated POST request. (2) The Auth & RBAC Service validates the analyst's session token before the request is forwarded. (3) The AI Analysis Engine extracts lexical and structural features from the URL and obtains a fast verdict from the local PhishGuard/random-forest models. (4) In parallel, the engine calls the Gemini API with the URL and the local verdict to generate a short, human-readable rationale explaining which features drove the score. (5) The Threat Intel Service independently queries VirusTotal for the URL or its resolved domain, returning a multi-engine detection ratio. (6) The AI Analysis Engine merges the local score, the Gemini rationale, and the VirusTotal ratio into a single risk verdict, which is returned to the dashboard for display. (7) If the merged verdict exceeds the configured severity threshold, the Alerts & Webhook Service creates an alert and dispatches it to the configured Discord or Slack channel, while the full analysis, including intermediate scores and the rationale text, is written to the audit log store for later review. This scenario illustrates how the architecture's service boundaries (Section III) translate directly into the stage-by-stage flow described above, and how explanation and alerting are produced alongside the verdict rather than after it.

Table 1: Mapping of Research Gaps Identified in Related Work to CyberShield AI Design Decisions

Gap Identified in Related Work	Corresponding Design Decision in CyberShield AI
Detection stops at classification; no coupling to real-time alerting [1, 3, 4]	A dedicated Alerts & Webhook Service categorizes every verdict by severity and dispatches it to Discord/Slack immediately after analysis.
High accuracy does not translate into user trust or detection competence [2, 9]	Every phishing and malware verdict is paired with a Gemini-generated XAI rationale alongside the local model's score, not a bare numeric output.
Cross-dataset and cross-domain generalization remains unresolved for single classifiers [1, 8]	The AI Analysis Engine combines local ML inference with an independent Gemini cross-check and VirusTotal's multi-engine consensus rather than relying on one static model.
Static or dynamic analysis alone is insufficient for fileless and evasive threats [7, 5]	The Threat Intel Service aggregates verdicts from more than seventy antivirus engines via VirusTotal, approximating multi-perspective analysis without requiring in-house forensic tooling.
Human and organizational factors are measurable but decoupled from technical tooling [10, 12]	Role-based access (User vs. SOC Admin) and a dedicated audit log store make detection outcomes and system usage visible, reviewable, and attributable.
Economic justification for security tooling is treated separately from detection capability [11]	Centralizing scanning, intelligence lookups, and alerting in one lightweight service reduces the tooling overhead of assembling separate point solutions, narrowing the investment case that FAIR-ROSI-style models must justify.
No single, lightweight, integrated platform for individuals and small teams [10, 9]	The full service-oriented architecture in Fig. 1 integrates authentication, detection, explanation, intelligence, and alerting into one deployable dashboard.

V. IMPLEMENTATION CONSIDERATIONS

A. Security and Access Control

Because the REST API Gateway is the single entry point for every client request, it is also the natural place to enforce security policy. The Auth & RBAC Service issues and validates session tokens and distinguishes at minimum two roles, User (Analyst) and SOC Admin, so that configuration actions such as editing webhook destinations or severity thresholds are restricted to administrators while scanning and viewing remain available to analysts. Centralizing authentication at the gateway, rather than in each downstream service, keeps the Auth & RBAC Service as the only component that needs to change if the authentication mechanism is upgraded, for example from session tokens to a federated identity provider. External credentials, the Gemini API key and the VirusTotal API key, are held server-side behind the gateway and are never exposed to the client, so a compromised browser session cannot be used to exhaust or misuse either external quota.

B. API and Data Design

The internal API is organized by resource rather than by screen, with endpoint groups for dashboard metrics, phishing scans, malware scans, network monitor data, threat-intelligence queries, alert management, and settings. This resource-oriented design lets the Web Dashboard compose views from multiple endpoints without the backend needing to know how the frontend will arrange them, and lets future clients, for example a command-line tool or a mobile companion app, reuse the same API surface. Firestore is used for data that must persist across sessions (user accounts, saved settings, alert history), paired with an in-memory cache for fields that are read far more often than they change, such as the current dashboard summary, so that repeated polling does not translate into repeated database reads.

C. Scalability and Extensibility

Because the AI Analysis Engine, Threat Intel Service, Auth & RBAC Service, and Alerts & Webhook Service communicate only through the REST API Gateway, each can be scaled or replaced independently. A production deployment under higher load could run multiple instances of the AI Analysis Engine behind the gateway without changing the Auth & RBAC Service, or add a new intelligence source to the Threat Intel Service without touching the alerting path. The same boundary makes it straightforward to extend the model layer,

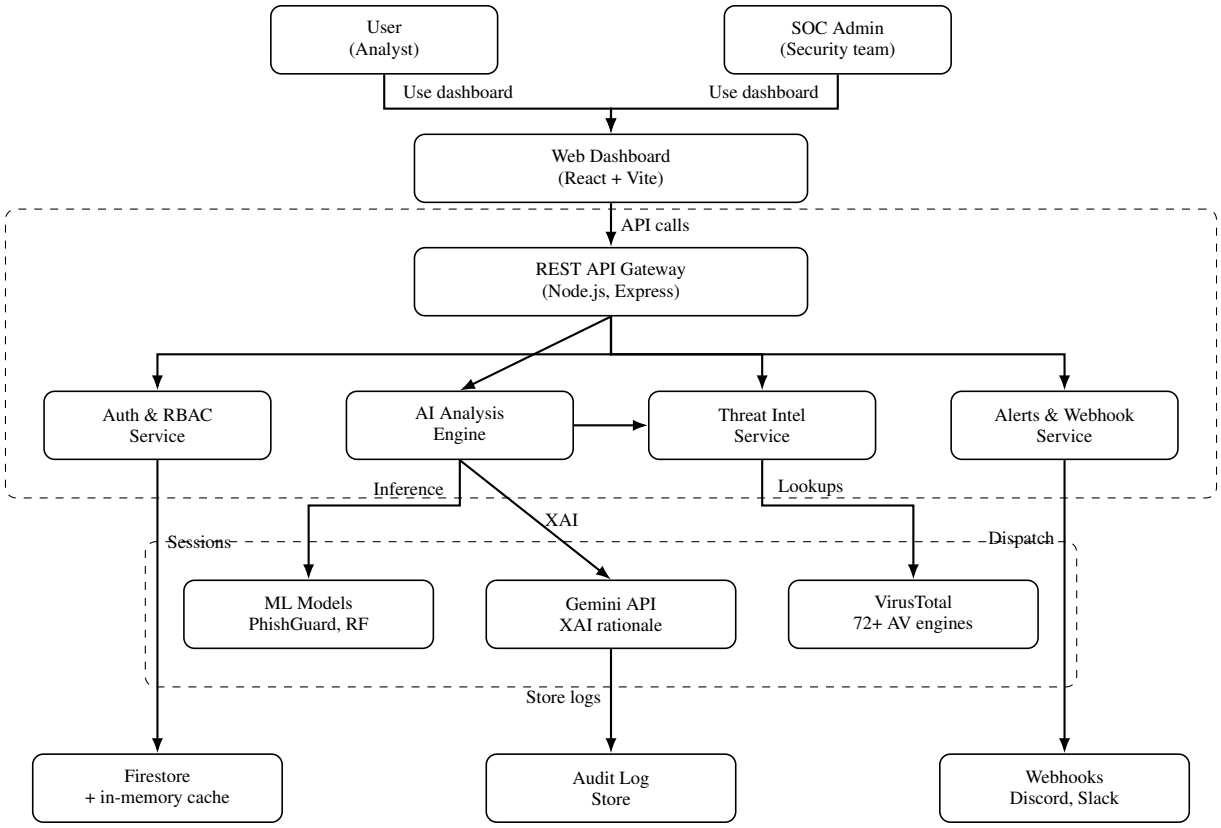


Figure 1: Proposed architecture of CyberShield AI. Users and SOC administrators interact with a React/Vite web dashboard, which calls a Node.js/Express REST API gateway. The gateway dispatches requests to an Auth & RBAC service, an AI analysis engine, a threat-intel service, and an alerts/webhook service; the AI and threat-intel services draw on ML models, the Gemini API, and VirusTotal, while sessions, logs, and alerts are persisted to Firestore, an audit log store, and Discord/Slack webhooks respectively.

for example swapping the random-forest baseline for one of the higher-accuracy classifiers surveyed in Section II, without changes outside the AI Analysis Engine itself.

D. Deployment and Observability

Each service in Fig. 1 is designed to run as an independently deployable process, so a containerized deployment (one container per service, orchestrated with a tool such as Docker Compose or Kubernetes) maps directly onto the architecture without further decomposition. Because every request already passes through the REST API Gateway and every AI verdict is written to the audit log store, the same two components double as natural points to attach operational monitoring: request latency and error rate can be measured at the gateway, and model-verdict volume, severity distribution, and Gemini/VirusTotal call latency can be derived directly from the audit log, giving the team a baseline for both system health and detection activity without instrumenting additional infrastructure. Centralized structured logging (for example, forwarding each service's logs to a single aggregator) would extend this further, letting the audit trail used for security review double as the primary signal for operational debugging.

VI. DISCUSSION

The architecture reflects three design choices directly informed by the gaps identified in Section II. First, explanation is treated as a first-class output rather than an afterthought: every phishing and malware verdict is paired with a Gemini-generated XAI rationale alongside the local model's score, addressing the finding that raw classification accuracy does not by itself build user trust or detection competence [2, 9]. Second, detection is coupled directly to alerting: rather than stopping at classification, detected threats are categorized by severity by a dedicated Alerts & Webhook Service and dispatched to Discord or Slack, closing the fragmentation between detection and action observed across the reviewed phishing and malware literature. Third, the system separates concerns into independent services behind a single API gateway, so that authentication, analysis, threat intelligence, and notification can each be scaled, replaced, or extended, such as upgrading the local ML models or adding further intelligence sources, without restructuring the rest of the system.

The design also has corresponding limitations that follow directly from its service boundaries. The quality of the AI Analysis Engine's verdicts depends on the local ML models deployed behind it, which are not a replacement for the spe-

cialized, dataset-validated classifiers surveyed in Section II unless trained and evaluated to a comparable standard; the Threat Intel Service's accuracy is bounded by VirusTotal's own coverage and update latency; and the audit log store's usefulness for incident response depends on retention policy and access controls that are outside the scope of the architecture described here. Realizing the full benefit of the design in a production deployment would require validating the local models against the benchmarks reviewed in Section II, hardening the Auth & RBAC Service for multi-tenant use, and defining retention and access policies for the Firestore and audit log stores.

VII. CONCLUSION AND FUTURE WORK

This paper presented CyberShield AI, a web-based cybersecurity monitoring dashboard that combines real-time telemetry visualization, ML-based phishing and malware scanning, and LLM-assisted explanation through the Google Gemini API, with threat-intelligence lookups and configurable webhook-based alerting. Positioned against recent research on phishing detection, malware detection, and human-centered cybersecurity, the system's service-oriented architecture directly addresses the recurring gap between high offline detection accuracy and the real-time, explainable, and actionable tooling needed by analysts and small teams. The mapping in Table 1 shows that each major gap identified in the reviewed literature, fragmented alerting, unexplained verdicts, single-model generalization limits, insufficient multi-perspective analysis, and the separation of human, organizational, and economic factors from technical tooling, corresponds to a specific element of the proposed architecture, indicating that the design is grounded in evidence from current research rather than in an arbitrary feature list.

The present work is a system architecture and design paper rather than an empirical evaluation, and several directions remain for future work before the system could be relied upon in a production SOC setting:

1. **Model validation and benchmarking.** The local PhishGuard and random-forest models should be trained and evaluated on the same public datasets used in Section II (e.g., PhiUSIIL, GramBeddings, BIG2015, Malimg) so that their accuracy, false-positive rate, and cross-dataset generalization can be reported and compared directly against the benchmarks surveyed in this paper, rather than assumed.
2. **Production-grade persistence.** The current Firestore and audit-log design should be extended with defined retention periods, backup procedures, and field-level access controls, and evaluated under realistic write volumes to confirm the in-memory cache strategy holds under concurrent multi-analyst use.
3. **Expanded threat-intelligence coverage.** Beyond VirusTotal, the Threat Intel Service could integrate additional open feeds (e.g., CVE/NVD advisories, open-source phishing block-lists) so that lookups are not bounded by a single provider's coverage and update latency.
4. **Hardened multi-tenant access control.** The Auth & RBAC Service should be extended from the two roles described in Section V to support organization-level tenancy, federated single sign-on, and fine-grained permissions (e.g., read-only auditor roles), which are necessary before the system could be offered to multiple independent teams.
5. **Broader alerting and response integration.** Webhook delivery to Discord and Slack could be extended toward ticketing systems and security-orchestration (SOAR) platforms, enabling an alert to trigger a tracked response workflow rather than only a chat notification.
6. **Explainability evaluation.** The quality of the Gemini-generated XAI rationale should be assessed directly, for example by measuring whether analysts presented with the rationale make faster or more accurate triage decisions than analysts shown a bare score, following the human-centered evaluation approach used in the awareness and training literature reviewed in Section II [9, 12].
7. **Usability study with SOC analysts.** A structured usability evaluation with practicing analysts or students in a security operations course would validate whether the dashboard's information hierarchy and alert triage flow actually reduce time-to-decision relative to using the underlying scanners and intelligence sources separately.
8. **Cost and investment analysis.** Applying an economic model such as FAIR-ROSI [11] to a deployed instance of CyberShield AI would quantify whether consolidating detection, explanation, and alerting into one platform reduces total security tooling cost relative to the point solutions it replaces.
9. **Continuous model monitoring and retraining.** Once the local ML models are deployed on live traffic, a feedback loop that logs analyst-confirmed true and false positives from the audit log store could be used to detect model drift over time and trigger periodic retraining, keeping detection accuracy aligned with the benchmarks established during initial validation.
10. **Operational observability at scale.** The logging and monitoring points identified in Section V-D (gateway latency, audit-log-derived verdict and severity metrics) should be connected to a dashboard or alerting system of their own, so that degradation in the AI Analysis Engine or Threat Intel Service is caught operationally before it affects detection quality.

Taken together, these directions would move CyberShield AI toward a validated, production-ready platform while preserving its core principle: delivering real-time detection, explanation, and alerting together.

REFERENCES

[1] M. Misiek and T. Hyla, "XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation," *IEEE Access*, vol. 14, 2026, doi: 10.1109/ACCESS.2026.3672690.

[2] T. Koide, H. Nakano, and D. Chiba, "ChatSpamDetector: Large Language Model-Based Phishing Email Detection," *IEEE Access*, vol. 14, 2026, doi: 10.1109/ACCESS.2026.3704566.

[3] L. Aldhuwayhi and A. Alromih, "A Dynamic Phishing URL Detection System Integrating Advanced Machine Learning and Crowdsourced Threat Intelligence," *IEEE Access*, 2026, doi: 10.1109/ACCESS.2026.3700833.

[4] J. D. Duarte, P. Chagas Junior, J. P. Javidi da Costa, E. J. da Costa, L. Peotta de Melo, R. Rabelo Nunes, C. G. V. N. Soares, and T. E. da Cunha Silva, "Machine Learning for Early Detection of Phishing URLs in Parked Domains: An Approach Applied to a Financial Institution," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3599454.

[5] A. Elgarduh, A. Zainal, F. A. Ghaleb, S. N. Qasem, A. M. Albarrak, and F. Saeed, "Hybrid Feature Image Mapping (HFIM): A Multi-Channel Image Approach for Malware Detection Using Static and Dynamic Features," *IEEE Access*, 2026 (in press).

[6] R. Gutierrez, W. Villegas-Ch., L. Naranjo Godoy, A. Mera-Navarrete, and S. Luján-Mora, "Application of Deep Learning Models for Real-Time Automatic Malware Detection," *IEEE Access*, vol. 12, 2024, doi: 10.1109/ACCESS.2024.3436588.

[7] I. Kara and K. Yılmaz, "HAP-Analyzer: A Hybrid Static, Dynamic, and Memory Analysis Approach for Fileless Malware," *IEEE Access*, vol. 14, 2026, doi: 10.1109/ACCESS.2026.3659346.

[8] M. Alshoulie and A. Mehmood, "Deep Learning Approaches for Malware Detection: A Comprehensive Review of Techniques, Challenges, and Future Directions," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3582875.

[9] R. Almarwani, M. Almarwani, and F. Almarwani, "AI-Driven Synthetic Threats in Cybersecurity: A User-Centered Framework for Awareness, Detection, and Protective Behavior," *IEEE Access*, vol. 14, 2026, doi: 10.1109/ACCESS.2026.3673980.

[10] K. Khattak, Sabahat, N. Iqbal, Zeeshan, and J. Choi, "Cyber Security Challenges: A Strategic Model for the Software Development Organizations," *IEEE Access*, vol. 14, 2026, doi: 10.1109/ACCESS.2026.3695471.

[11] Y. He, T. Xin, and C. Luo, "Enhancing Cybersecurity Investment with FAIR-ROSI: A Responsible Cybersecurity Approach to Digital Society," *Information Systems Frontiers*, vol. 28, pp. 697–712, 2026.

[12] T. Abril, P. Gamito, C. da Motta, J. Oliveira, F. Dias, F. Pinto, et al., "Exploring a Novel Approach to Cybersecurity: The Role of Ecological Simulations on Cybersecurity Risk Behaviors," *Virtual Reality*, vol. 29, art. 150, 2025.